

Esame OS Internals (Cabodi)

Data: 05/07/2021

Nota Generale: TUTTE LE RISPOSTE SI / NO DEVONO ESSERE MOTIVATE. QUANDO I RISULTATI SONO NUMERI, SONO RICHIESTI SIA IL RISULTATO FINALE SIA I RELATIVI PASSI INTERMEDI (O FORMULE).

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Inverted Page Table e Dimensionamento)

Contesto:

A) Si considerino le affermazioni che seguono, a proposito di possibili vantaggi e svantaggi di una inverted page table (IPT), rispetto a una tabella delle pagine standard (PT). Si dica di ognuna se sia vera o falsa, motivando la risposta:

- **Vantaggi:** L'IPT permette di risparmiare memoria:

1. Si risparmia sempre memoria.
2. Dipende dalle dimensioni della RAM, dal numero di processi e dal loro spazio di indirizzamento virtuale.
3. Si risparmia sempre memoria quando lo spazio di indirizzamento di un processo è maggiore della dimensione della RAM.
4. Si può risparmiare anche in casi in cui lo spazio di indirizzamento di ogni processo è inferiore alla dimensione della RAM.

- **Svantaggi:** L'IPT è lenta, perché non garantisce accesso diretto ma occorre una ricerca:

1. La chiave di ricerca è il frame.
2. La chiave di ricerca è la pagina.
3. La chiave di ricerca è la coppia (pid, frame).
4. La chiave di ricerca è la coppia (pid, pagina).
5. Per migliorare le prestazioni si sostituisce la IPT con una tabella di HASH.
6. Per migliorare le prestazioni si aggiunge alla IPT una tabella di HASH.

B) Calcoli di memoria: Sia dato un processo avente spazio di indirizzamento virtuale di 48 GB, in un sistema dotato di 16GB di RAM, con architettura a 64 bit (in cui si indirizza il Byte) e gestione della memoria paginata (pagine/frame da 4KB) per i soli processi user, quindi non per il kernel. Si supponga che 4GB di RAM siano allocati in modo statico al kernel. Si vogliono confrontare una soluzione basata su tabella delle pagine standard (una tabella per ogni processo) e una basata su IPT. Si calcolino:

B1) Le dimensioni della PT (a un solo livello) per il processo e della IPT. Si ipotizzi che il pid di un processo possa essere rappresentato su 12 bit. Si utilizzino 28 bit per gli indici di pagina e/o di frame (nella PT o nella IPT) e si tenga conto che, per allineamento, una cella di IPT o PT può solo essere di 32 o 64 bit.

- B2)** Si dica infine, utilizzando la IPT proposta (12 bit di pid, 28 bit per un indice di pagina/frame), quale è la dimensione massima possibile per lo spazio di indirizzamento virtuale di un processo.
-

Domanda 2 (RAID e Affidabilità)

Contesto: Sia dato un disco organizzato con struttura RAID. Sono dati il tempo medio tra guasti (MTTF: Mean Time To Failure), il tempo medio per riparazione (MTTR: Mean Time To Repair), e il tempo medio tra perdite di dato (MTTDL: Mean Time To Data Loss).

Si risponda alle seguenti domande:

A) E' possibile che (motivare le risposte):

A1) $MTTR > MTTF$?

A2) $MTTDL > 10 \cdot MTTF$?

A) Si consideri una struttura RAID con 2 dischi in configurazione "mirrored". Se ognuno dei dischi può guastarsi in modo indipendente dall'altro, con $MTTF = 20000$ ore, quanto deve essere MTTR per garantire $MTTDL > 100 \cdot MTTF$?

Domanda 3 (I/O, DMA e Cycle Stealing)

Contesto: Si consideri il problema della gestione di una richiesta di I/O a blocchi realizzato mediante DMA.

Si risponda alle seguenti domande:

A) Che cosa si intende, in questo contesto, con il termine "cycle stealing"?

A) Perché il trasferimento in DMA è vantaggioso rispetto all'I/O programmato?

A) Immaginando di dover trasferire 10MB di dati da disco a memoria RAM, quanti Byte transitano sul bus dati della RAM nei due casi (trasferimento in DMA e I/O programmato)? (Motivare).

A) Qualora si volesse usare per l'I/O, mediante DMA, un buffer in memoria Kernel, invece di usare direttamente la sorgente/destinazione in memoria user, il DMA controller farebbe più o meno trasferimenti, oppure lo stesso numero, rispetto alla versione senza buffer? (Motivare).

2 SECTION 2: OS161

Domanda 4 (Copia di un Address Space in OS161)

Contesto: Sia dato un sistema OS161. Si consideri la funzione `as_copy`, avente come obiettivo quello di "duplicare" un address space, cioè creare un nuovo address space a partire da uno esistente di cui si fa una copia. La `as_copy` potrebbe essere usata, ad esempio, per realizzare la system call `fork`, che genera un processo child a partire da un processo (parent) esistente. Si propone una possibile implementazione della `as_copy`:

```
int as_copy(struct addrspace *old, struct addrspace **ret) {
    struct addrspace *new;
    dumbvm_can_sleep();
    new = as_create();

    new->as_vbase1 = old->as_vbase1;
    new->as_npages1 = old->as_npages1;
    new->as_vbase2 = old->as_vbase2;
    new->as_npages2 = old->as_npages2;
    new->as_pbase1 = old->as_pbase1;
    new->as_pbase2 = old->as_pbase2;
    new->as_stackpbase = old->as_stackpbase;

    memmove((void *)PADDR_TO_KVADDR(new->as_pbase1),
            (const void *)PADDR_TO_KVADDR(old->as_pbase1),
            old->as_npages1 * PAGE_SIZE);

    memmove((void *)PADDR_TO_KVADDR(new->as_pbase2),
            (const void *)PADDR_TO_KVADDR(old->as_pbase2),
            old->as_npages2 * PAGE_SIZE);

    memmove((void *)PADDR_TO_KVADDR(new->as_stackpbase),
            (const void *)PADDR_TO_KVADDR(old->as_stackpbase),
            DUMBVM_STACKPAGES * PAGE_SIZE);

    *ret = new;
    return 0;
}
```

Si risponda alle seguenti domande:

- B) Perché il parametro `old` ha un solo asterisco mentre `ret` ne ha due?
- B) La funzione `as_create` alloca RAM per i due segmenti (testo e dati) e per lo stack del nuovo address space?
- B) La funzione contiene volutamente degli errori. Si tratta di errori semantici, legati all'obiettivo della funzione, quindi non di errori sintattici: si individuino tali errori e si spieghi perché lo sono.
- B) La funzione `memmove` viene qui utilizzata per copiare il contenuto di una parte di RAM fisica in un'altra parte di RAM fisica: che cosa realizza `PADDR_TO_KVADDR` e perché viene utilizzata in questo contesto, invece di chiamare semplicemente, ad esempio: `memmove((void*)(new->as_pbase1), (const void*)(old->as_pbase1), old->as_npages1*PAGE_SIZE);`?

Domanda 5 (Implementazione System Call `sys_exit`)

Contesto: In un sistema OS161, si vuole realizzare il supporto per la system call `SYS_exit` (che corrisponde alla `void exit(int status);`). NON SI INTENDE ANCORA SUPPORTARE LA `waitpid`, ma unicamente il salvataggio dello stato (`status`), la terminazione del thread e il rilascio dell'address space.

Si risponda alle seguenti domande:

- B) Si supponga che la funzione `syscall()`, in corrispondenza al valore `SYS_exit` in `callno` chiami la funzione `my_sys_exit()`, si definisca il prototipo di tale funzione e se ne scriva la chiamata (con parametri attuali) in `syscall()`.

```
/* METTERE TIPO DI RITORNO */
my_sys_exit(/* METTERE PARAMETRI QUI */);

void syscall(struct trapframe *tf) {
    /* ... */
    callno = tf->tf_v0;
    /* ... */
    switch (callno) {
        /* ... */
        case SYS_exit:
            err = my_sys_exit(/* METTERE PARAMETRI QUI */);
            break;
    }
}
```

- B) Si osservi il seguente frammento di codice (incompleto) della `my_sys_exit()`. Si dica da dove può o deve provenire il valore `status`, e a che variabile o campo di struct va assegnato (dire quindi cosa va sostituito ai ...). Si identifichino e si commentino gli errori o problemi contenuti nelle successive istruzioni:

```
my_sys_exit (...) {
    ... = status;
    thread_exit(curthread);
    proc_destroy(curproc);
    as_destroy(curproc->p_addrspace);
}
```